

Один цикл, который нужно запомнить

КОНТРАКТ ДАННЫЕ REQUEST RESPONSE STATE CLEANUP DIAGNOSE

1. Понять method/path/auth/request/response.
2. Создать контролируемые данные.
3. Отправить request.
4. Проверить transport, structure и business data.
5. После mutation прочитать сохранённое состояние.
6. Удалить созданное даже после падения.
7. Оставить данные для поиска причины.

Роли инструментов

REST Assured	HTTP request/response.
Hamcrest	Короткие inline body checks.
Jackson	JSON ↔ Java DTO.
AssertJ	Связанные данные и DTO.
JUnit	Запуск, lifecycle, parameterization.

Что доказывает API-тест

Transport	Endpoint отвечает правильным status/type.
Structure	Поля, types и вложенность допустимы.
Business	Вернулись правильные значения.
State	Изменение реально сохранилось.
Security	Запрещённая операция отклоняется.

P0 Без этого тест не рабочий

- Конкретный status, а не «любой 2xx».
- Ключевые значения связаны с request.
- Собственный id вместо общей записи.
- Read-back после POST/PUT/PATCH/DELETE.
- Отказ без auth для защищённой операции.
- Cleanup и условное логирование.
- Каждый тест запускается отдельно.

Почему status 200 недостаточно

Можно получить 200 с пустым body, чужой записью, неверной ценой или несохранённым PATCH. Status — первый слой, а не финал.

P1 Ежедневная работа

DTO/Jackson, JsonPath, specifications, schema, AssertJ для DTO, JUnit parameterization и lifecycle. Они усиливают P0, но не заменяют его.

Общий учебный сервис

Restful Booker периодически сбрасывает данные. Нельзя полагаться на постоянный id/размер списка. Создавайте своё и очищайте после теста.

P2 После уверенной базы

Custom filters, сложный GPath, parallel suite, расширенный HTTP config, retries, Allure/CI integrations.

Редкий метод библиотеки не компенсирует отсутствие state check.

Цель

Взять новый endpoint, самостоятельно выбрать данные и проверки, собрать устойчивый сценарий и объяснить причину падения.

7 вопросов до кода

1. HTTP method?
2. Path?
3. Path/query params?
4. Auth?
5. Request headers?
6. Request body?
7. Response и способ подтверждения эффекта?

Endpoints Restful Booker

GET /ping	Health, 201.
GET /booking	Список id/filters.
GET /booking/{id}	Один booking.
POST /booking	Создание.
POST /auth	Token.
PUT /booking/{id}	Полная замена, auth.
PATCH /booking/{id}	Частичное изменение, auth.
DELETE /booking/{id}	Удаление, auth.

Анатомия request

```
given()
  .baseUrl(BASE_URL)
  .accept(ContentType.JSON)
  .contentType(ContentType.JSON)
  .pathParam("id", bookingId)
  .queryParams("firstname", "Ivan")
  .cookie("token", token)
  .body(requestBody)
  .when()
  .put("/booking/{id}");
```

baseUrl	Сервер.
accept	Формат response.
contentType	Формат request body.
pathParam	Конкретный ресурс.
queryParams	Фильтр.
cookie/auth	Полномочия.

Map, DTO или JSON string?

Map	Маленький PATCH/обучение JSON. Опечатки в ключах.
DTO	Основной вариант: типы, IDE, reuse, сравнение.
String	Короткий сырой/негативный payload.

```
BookingRequest body = new BookingRequest(
    "Ivan", "Student", 150, true,
    new BookingDates("2026-08-01", "2026-08-05"),
    "Breakfast");

given().contentType(ContentType.JSON).body(body);
```

Path и query — разный смысл

```
// конкретный ресурс
.pathParam("id", bookingId)
.get("/booking/{id}")

// фильтр списка
.queryParam("firstname", firstName)
.get("/booking")
```

Правило

Я могу вслух прочитать: куда, каким методом, с какими полномочиями и данными идёт request.

Ошибки request

- Перепутаны Accept и Content-Type.
- URL склеен строкой.
- Token попал в лог.
- DTO имеет неверные JSON names.
- Mutation идёт по чужому id.
- Request не соответствует документации.

Пять слоёв сверху вниз

1. Transport	Status, content type, важные headers.
2. Structure	Поля, JSON types, nesting, schema.
3. Business	Значения относятся к request.
4. Saved state	GET подтверждает mutation.
5. Unchanged state	PATCH не испортил остальное.

Response mutation не доказывает state

Недостаточно

```
PATCH ...  
  .then()  
  .statusCode(200);
```

Нужно

```
PATCH → 200  
GET /id → saved  
assert saved
```

P0 Конкретный transport

```
response.then()  
  .statusCode(200)  
  .contentType(ContentType.JSON);
```

200, 201, 204, 400, 403 и 404 — разные результаты. «status < 500» обычно слишком слабо.

Разные mutation — разные ожидания

PUT	Saved DTO полностью равен updated.
PATCH	Changed field + preserved fields.
DELETE	GET после удаления → 404.

Structure + business

```
response.then()  
  .body("bookingid", greaterThan(0))  
  .body("booking.firstname",  
    equalTo(expected.firstname()))  
  .body("booking.bookingdates.checkin",  
    equalTo(expected.bookingdates().checkin()));
```

Минимум create-теста

1. POST вернул status и JSON.
2. bookingid положительный.
3. Response связан с request.
4. GET по id вернул booking.
5. DTO совпал с expected.
6. id зарегистрирован для cleanup.

Schema — один слой

JSON может соответствовать schema и содержать чужой firstname. Structure не заменяет business.

Проверяйте риск

- Идентификатор создан.
- Ответ относится к request.
- Ключевое поле корректно.
- Вложенный объект не потерян.
- Mutation сохранилась.

Самопроверка

Мой тест доказывает фактическое состояние или только красивый response?

Почему нельзя менять «первый id»

- Он может принадлежать другому тесту.
- База может сброситься.
- Параллельный пользователь изменит/удалит запись.
- Порядок общего списка нестабилен.

Read-only знакомство может извлечь текущий id. Mutation — только на своём.

Уникальные данные

```
String firstName = "ApiTest-" + UUID.randomUUID();
```

- Префикс показывает владельца.
- UUID снижает конфликт.
- Значение нужно сохранить в отчёт.
- Остальные поля валидны и понятны.

Создать и извлечь id

```
BookingRequest expected = uniqueBooking("Create");

int bookingId = given()
    .spec(BASE_SPEC)
    .contentType(ContentType.JSON)
    .body(expected)
    .when()
    .post("/booking")
    .then()
    .statusCode(200)
    .extract().path("bookingid");

createdBookingIds.add(bookingId);
```

Cleanup через JUnit

```
@AfterEach
void cleanup() {
    for (int id : createdBookingIds) {
        deleteBookingIfPresent(id);
    }
    createdBookingIds.clear();
}
```

Регистрируйте id сразу после создания, а не в конце сценария.

Страховка try/finally

```
Integer id = null;
try {
    id = createBooking();
    // GET → PATCH → GET → DELETE → GET 404
} finally {
    if (id != null) deleteBookingIfPresent(id);
}
```

Cleanup терпим к уже удалённому resource и не скрывает main failure.

Прочитать saved DTO

```
BookingResponse actual = given()
    .spec(BASE_SPEC)
    .pathParam("id", bookingId)
    .when()
    .get("/booking/{id}")
    .then()
    .statusCode(200)
    .extract().as(BookingResponse.class);

assertThat(actual)
    .usingRecursiveComparison()
    .isEqualTo(expected);
```

Инварианты независимости

- Создаёт всё, что изменяет.
- Удаляет только своё.
- Не зависит от порядка.
- Параллельные не смешиваются.
- Запускается отдельно.

Дерево решения

1 значение дальше	extract().path
2-4 простых поля	Hamcrest/JsonPath
Объект целиком	DTO + AssertJ
Форма/types	Schema дополнительно
Список/filter	JsonPath или DTO list

DTO + AssertJ

```
BookingResponse actual =
    response.as(BookingResponse.class);

assertThat(actual)
    .usingRecursiveComparison()
    .isEqualTo(expected);
```

Основной вариант для полного booking: типы, IDE и понятные различия.

Inline Hamcrest

```
response.then()
    .statusCode(200)
    .body("firstname", equalTo(expected.firstname()))
    .body("totalprice", equalTo(expected.totalprice()))
    .body("bookingdates.checkin",
        equalTo(expected.bookingdates().checkin()));
```

Компактно для малого контракта. Десять полей превращаются в стену JsonPath.

SoftAssertions для PATCH

```
SoftAssertions.assertSoftly(softly -> {
    softly.assertThat(saved.lastname())
        .as("changed").isEqualTo(changed);
    softly.assertThat(saved.firstname())
        .as("preserved
name").isEqualTo(original.firstname());
    softly.assertThat(saved.bookingdates())
        .as("preserved
dates").isEqualTo(original.bookingdates());
});
```

Extraction и JsonPath

```
int id = response.then()
    .statusCode(200)
    .extract().path("bookingid");

String first = response.jsonPath().getString("firstname");
int price = response.jsonPath().getInt("totalprice");
```

Extraction связывает шаги flow; JsonPath достаёт точечные значения.

Schema + business

```
response.then()
    .body(matchesJsonSchemaInClasspath(
        "schemas/lesson01/booking-schema.json"))
    .body("firstname", equalTo(expected.firstname()));
```

Ловит	Required, type, nesting, format.
Не ловит	Чужие values, state, auth.

Антипаттерн

Извлечь десять полей вручную и собрать объект. Если response — связанный объект, десериализуйте сразу.

Правило читаемости

Способ проверки должен делать контракт очевиднее. Иначе извлеките DTO или упростите.

RequestSpecification

```
RequestSpecification baseSpec =  
    new RequestSpecBuilder()  
        .setBaseUri(BASE_URL)  
        .setAccept(ContentType.JSON)  
        .addFilter(new ErrorLoggingFilter())  
        .build();
```

Уместно

- base URI и safe headers;
- Accept/Content-Type группы;
- timeouts/config;
- failure logging;
- technical auth клиента.

Разделение слоёв

Tests	Сценарий, expected, assertions.
Client	Выполнение endpoint без business asserts.
Models	Request/response DTO.
Fixtures	Валидные уникальные данные.
Specs/config	URL, headers, HTTP settings, logging.

Логировать при провале

```
given()  
    .log().ifValidationFails()  
    .when()  
    .get("/booking/{id}", id)  
    .then()  
    .log().ifValidationFails();
```

Глобальный log().all() создаёт шум и может раскрыть token/password.

Не прячьте в спец

- Данные конкретного теста.
- Business assertions.
- Status для разных исходов endpoint.
- Изменяемый глобальный body.
- Секреты в открытом логировании.

Минимум диагностики

1. Environment/base URL.
2. Method, path, params.
3. Safe request headers/body.
4. Response status/headers/body.
5. Созданный id и unique data.
6. Expected/actual и шаг flow.

ResponseSpecification

```
ResponseSpecification okJson =  
    new ResponseSpecBuilder()  
        .expectStatusCode(200)  
        .expectContentType(ContentType.JSON)  
        .build();  
  
response.then()  
    .spec(okJson)  
    .body("firstname", equalTo(expected.firstname()));
```

Классифицируйте

Product	PATCH изменил лишнее.
Test	Неверный path/expected.
Data	Чужой id.
Environment	DNS/timeout/down.
Contract	Docs ≠ implementation.

Получить и использовать token

```
String token = given()
    .spec(BASE_SPEC)
    .contentType(ContentType.JSON)
    .body(new AuthRequest("admin", "password123"))
    .when()
    .post("/auth")
    .then()
    .statusCode(200)
    .body("token", notNullValue())
    .extract().path("token");

given().cookie("token", token);
```

PATCH: changed + preserved

```
Map<String, Object> patch =
    Map.of("lastname", changedLastName);

given()
    .spec(BASE_SPEC)
    .cookie("token", token)
    .contentType(ContentType.JSON)
    .body(patch)
    .when()
    .patch("/booking/{id}", bookingId)
    .then()
    .statusCode(200);

BookingResponse saved =
    getBooking(bookingId).as(BookingResponse.class);
```

P0 Negative auth

- Неверный password → Bad credentials.
 - Неверный username → Bad credentials.
 - PUT/PATCH/DELETE без token → 403.
- Happy path не доказывает защиту endpoint.

DELETE: response + state

```
given()
    .spec(BASE_SPEC)
    .cookie("token", token)
    .pathParam("id", bookingId)
    .when()
    .delete("/booking/{id}")
    .then()
    .statusCode(201);

getBooking(bookingId).then().statusCode(404);
```

PUT: полная замена

1. Создать original.
2. Собрать полный updated DTO.
3. PUT с token.
4. Проверить response.
5. GET saved.
6. Recursive compare с updated.

Что забывают

- PUT отправляет неполный body.
- PATCH не подтверждается GET.
- Не проверены preserved fields.
- DELETE не подтверждён 404.
- Изменяется чужая запись.
- Token печатается в лог.

Секреты

Token, Authorization и password маскируются в CI/Allure. Failure logging безопаснее глобального.

Формула

before state → **authorized action** → **response** → **after state** → **invariants**.

JUnit формирует suite

@Test	Один independent scenario.
@ParameterizedTest	Один contract, таблица inputs.
@BeforeEach	Fixture/config.
@AfterEach	Cleanup resources.
@Tag	smoke/regression/api.

AssertJ проверяет Java DTO

```
BookingResponse actual =  
    response.as(BookingResponse.class);  
  
assertThat(actual)  
    .usingRecursiveComparison()  
    .isEqualTo(expected);
```

REST Assured проверил HTTP, Jackson создал DTO, AssertJ показал различия.

Invalid auth как data table

```
@ParameterizedTest  
@CsvSource({  
    "admin, wrong-password",  
    "wrong-user, password123",  
    "wrong-user, wrong-password"  
})  
void invalidAuth(String username, String password) {  
    Response r = auth(username, password);  
    assertThat(r.statusCode()).isEqualTo(200);  
    assertThat(r.jsonPath().getString("reason"))  
        .isEqualTo("Bad credentials");  
}
```

Soft assertions для связанных полей

```
SoftAssertions.assertSoftly(softly -> {  
    softly.assertThat(saved.lastname())  
        .as("changed").isEqualTo(changed);  
    softly.assertThat(saved.firstname())  
        .as("preserved").isEqualTo(original.firstname());  
    softly.assertThat(saved.totalprice())  
        .as("price").isEqualTo(original.totalprice());  
});
```

Что где проверять

REST Assured	Status/type/headers/small body.
AssertJ	DTO, lists, recursive/soft.
JUnit	Execution/lifecycle/data/report.

Не связывайте методы

Плохо

testUpdate использует static id из testCreate.

Хорошо

Каждый тест создаёт своё; CRUD живёт внутри одного метода.

Не делайте assert true

Проверяйте actual-данные типовыми assertions. Иначе теряется причина расхождения.

Параллельность начинается с дизайна

Unique data, локальный expected, нет static mutable id, cleanup только своего resource.

Единая картина

JUnit → REST Assured → Jackson → AssertJ → JUnit cleanup.

Финальный independent flow

1. Создать unique request DTO.
2. POST, transport/body, extract id.
3. Сразу зарегистрировать cleanup.
4. GET, DTO, recursive compare.
5. PATCH одного поля с token.
6. GET, changed + preserved.
7. DELETE с token.
8. GET → 404.
9. finally страхует cleanup.

Алгоритм нового endpoint

1. Выписать method/path/auth/request/response.
2. Определить риск и state transition.
3. Спроектировать independent data.
4. Вручную изучить real response.
5. Автоматизировать happy path.
6. Для mutation добавить read-back.
7. Добавить ценные negative cases.
8. Добавить cleanup/failure logs.
9. Только затем вынести reuse.
10. Запустить отдельно и в suite.

Скелет

```
BookingRequest initial = uniqueBooking("Final");
Integer id = null;
try {
    id = create(initial);           // POST
    assertBooking(id, initial);    // GET + AssertJ

    patchNeeds(id, "Lunch", token); // PATCH
    assertNeeds(id, "Lunch");       // GET

    delete(id, token);            // DELETE
    get(id).then().statusCode(404); // state
} finally {
    if (id != null) deleteIfPresent(id);
}
```

Checklist review

- ☐ Contract, не только доступность.
- ☐ Expected связан с request.
- ☐ Нет чужого/static id.
- ☐ Mutation подтверждена GET.
- ☐ PATCH проверяет preserved.
- ☐ Auth positive + negative.
- ☐ Cleanup переживает failure.
- ☐ Логи без секретов.
- ☐ Specs не скрывают business.
- ☐ Нет зависимости от порядка.

«Спокойно использую в работе»

Не нужно помнить каждый метод. Нужно без подсказки спроектировать данные, request, assertions, state verification, negative coverage, cleanup и диагностики.

Локализуйте шаг

POST	Payload/type/create contract.
GET 1	Persistence/DTO/data.
PATCH	Auth/partial update.
GET 2	Changed/preserved state.
DELETE/404	Deletion/state.
Cleanup	Infra; не скрывать main error.

Главный приоритет

Простой independent тест с read-back лучше сложного fluent-кода, проверяющего только status.

Дальше P2

OAuth, custom filters, parallel suite, service virtualization, CI secrets, Allure attachments, HTTP tuning — по потребности проекта.